实 验 报 告

实验名称：实验六 TopoICA 模型实现

课程名称：认知科学与类脑识别

实验地点：K2

实验日期：2024.10.18

班 级：2022 级人工智能班

姓 名：薛润泽

学 号：202200130077

一 实验目的：

加深对 TopoICA 模型的理解，能够使用 TopoICA 模型解决简单问题

二 实验环境：

Windows

三 实验内容：

根据 TopoICA 模型的相关知识，使用 Python 语言实现一个简单的 TopoICA 模型。

实验要求：

（1） 下载 MNIST 数据集。

（2） 构建 TopoICA 模型。

（3） 使用 MNIST 数据集中的训练集训练网络，使用测试集测试训练好的网络。

四 实验步骤：

1. 实验程序：

（1）导入 MNIST 数据库，加载数据

```
import numpy as np
import struct
import matplotlib.pyplot as plt

def load_mnist_images(filename):
    with open(filename, 'rb') as f:
        magic, num, rows, cols = struct.unpack(">IIII", f.read(16))
        images = np.fromfile(f, dtype=np.uint8).reshape(num, rows, cols)
        images = images.astype(np.float32) / 255.0  # 归一化到 [0, 1]
    return images

def load_mnist_labels(filename):
    with open(filename, 'rb') as f:
        magic, num = struct.unpack(">II", f.read(8))
        labels = np.fromfile(f, dtype=np.uint8)
    return labels

train_images = load_mnist_images('MNIST/raw/train-images-idx3-ubyte')
train_labels = load_mnist_labels('MNIST/raw/train-labels-idx1-ubyte')
test_images = load_mnist_images('MNIST/raw/t10k-images-idx3-ubyte')
test_labels = load_mnist_labels('MNIST/raw/t10k-labels-idx1-ubyte')
```

（2）定义激活函数 sigmoid 和 tanh，以及两个函数关于 x 的一阶导数

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return sigmoid(x) * (1 - sigmoid(x))

def tanh(x):
    return np.tanh(x)

def tanh_derivative(x):
    return 1 - np.tanh(x) ** 2
```

（3）定义 topoICA 的训练代码，其中 W 使用 He 方法进行初始化，将 W 矩阵权重初始值设置为一个服从以 0 为均值，标准差为 sqrt(2/n) 的正态分布。

在循环训练中，首先计算独立成分矩阵 S，再使用 tanh 激活函数计算输出值 Y。得到 W 和 Y 值后可以计算重构数据，从而得到重构误差 error。结合 tanh 导数计算出梯度 grad，并同时加以裁剪，防止梯度爆炸。得到方向梯度后使用梯度上升法更新权重。

```
def topoica_train(X, num_components, learning_rate=0.000005, iterations=1000, clip_value=5.0):
    np.random.seed(42)
    W = np.random.randn(num_components, X.shape[1]) * np.sqrt(2.0 / X.shape[1])  # He 初始化

    for i in range(iterations):
        S = np.dot(X, W.T)  # 独立成分
        Y = tanh(S)
        error = X - np.dot(Y, W)
        grad = np.dot(error.T, Y * tanh_derivative(S))
        grad = np.clip(grad, -clip_value, clip_value)  # 防止梯度爆炸
        W += learning_rate * grad.T

        if i % 100 == 0:
            loss = np.mean(np.abs(error))
            print(f"迭代 {i}: 损失 = {loss}")

    return W
```

（4）变化数据格式，并加以训练

```
56
57  train_images_flat = train_images.reshape(train_images.shape[0], -1)
58  test_images_flat = test_images.reshape(test_images.shape[0], -1)
59
60  num_components = 64
61  W = topoica_train(train_images_flat[:1000], num_components)
62
```

（5）可视化 TOPOICA 模型提取出的特征向量

```
def visualize_components(W, num_components, img_shape=(28, 28)):
    fig, axes = plt.subplots(8, 8, figsize=(10, 10))
    for i, ax in enumerate(axes.flat):
        if i < num_components:
            component_image = W[i].reshape(img_shape)
            ax.imshow(component_image, cmap='gray')
            ax.set_xticks([])
            ax.set_yticks([])
    plt.show()

visualize_components(W, num_components)
```

（6）得到权重矩阵后，可以根据数据构建出预测图像，并计算和真实数据相比的重建误差。

```
# 重建测试图像
def reconstruct_images(X, W):
    S = np.dot(X, W.T)
    Y = tanh(S)
    reconstructed_X = np.dot(Y, W)
    return reconstructed_X

# 计算重建误差
def calculate_reconstruction_error(original_X, reconstructed_X):
    return np.mean((original_X - reconstructed_X) ** 2)

reconstructed_test_images = reconstruct_images(test_images_flat, W)
reconstruction_error = calculate_reconstruction_error(test_images_flat, reconstructed_test_images)
print(f"测试集的重建误差为: {reconstruction_error}")
```

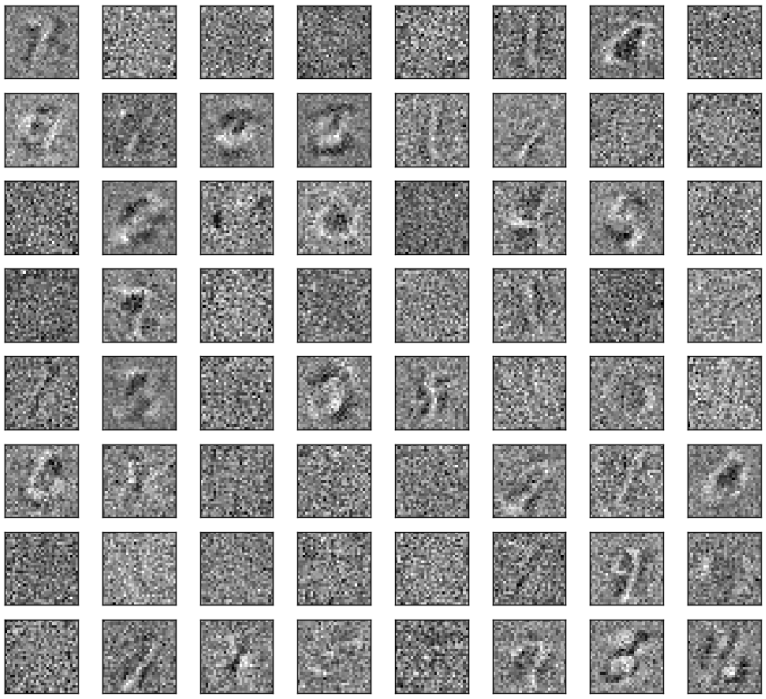
（7）将预测图像可视化

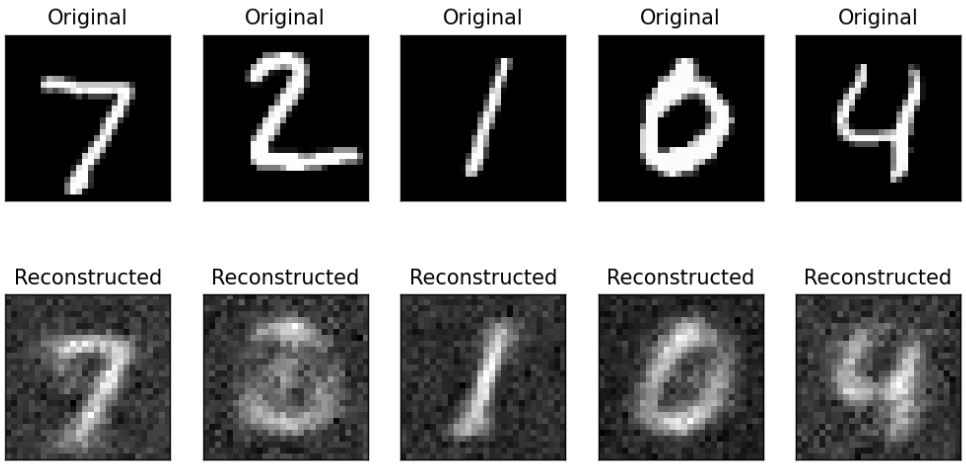
```
90  # 可视化原始和重建的测试图像
91  def visualize_reconstructed_images(original_images, reconstructed_images, num_images=5):
92      fig, axes = plt.subplots(2, num_images, figsize=(10, 5))
93      for i in range(num_images):
94          axes[0, i].imshow(original_images[i].reshape(28, 28), cmap='gray')
95          axes[0, i].set_title("Original")
96          axes[0, i].set_xticks([])
97          axes[0, i].set_yticks([])
98
99          axes[1, i].imshow(reconstructed_images[i].reshape(28, 28), cmap='gray')
100         axes[1, i].set_title("Reconstructed")
101         axes[1, i].set_xticks([])
102         axes[1, i].set_yticks([])
103     plt.show()
104
105 visualize_reconstructed_images(test_images_flat, reconstructed_test_images)
```

2. 程序实验结果：

（1）使用随机 1000 个样本，步长为 0.000005，迭代次数为 3000 结果如下：

```
PS D:\大学\大三上\认知科学与类脑识别\ex6> python -u "d:\大学\
训练集图片大小: (60000, 28, 28), 训练集标签大小: (60000,)
测试集图片大小: (10000, 28, 28), 测试集标签大小: (10000,)
迭代 0: 损失 = 0.21467224140131264
迭代 100: 损失 = 0.2101444397006033
迭代 200: 损失 = 0.1952955253960426
迭代 300: 损失 = 0.18049051503218533
迭代 400: 损失 = 0.17091943403300264
迭代 500: 损失 = 0.16571992469258637
迭代 600: 损失 = 0.16320168351829506
迭代 700: 损失 = 0.16165308848496088
迭代 800: 损失 = 0.1602233467683127
迭代 900: 损失 = 0.15867721350201774
迭代 1000: 损失 = 0.15697045045551625
迭代 1100: 损失 = 0.15512052240666901
迭代 1200: 损失 = 0.15316997303090674
迭代 1300: 损失 = 0.15115155517932197
迭代 1400: 损失 = 0.1490979467596196
迭代 1500: 损失 = 0.1470371719948469
迭代 1600: 损失 = 0.1449936229614772
迭代 1700: 损失 = 0.14298888276903157
迭代 1800: 损失 = 0.14103826984056775
迭代 1900: 损失 = 0.1391607433624741
迭代 2000: 损失 = 0.1373689493631167
迭代 2100: 损失 = 0.13566987033427352
迭代 2200: 损失 = 0.13406568686671394
迭代 2300: 损失 = 0.13255575281694934
迭代 2400: 损失 = 0.13114210774921709
迭代 2500: 损失 = 0.12982237773546895
迭代 2600: 损失 = 0.12858976457324758
迭代 2700: 损失 = 0.1274375472979027
迭代 2800: 损失 = 0.1263605475131842
迭代 2900: 损失 = 0.12535213007546922
测试集的重建误差为: 0.04033101224972884
```

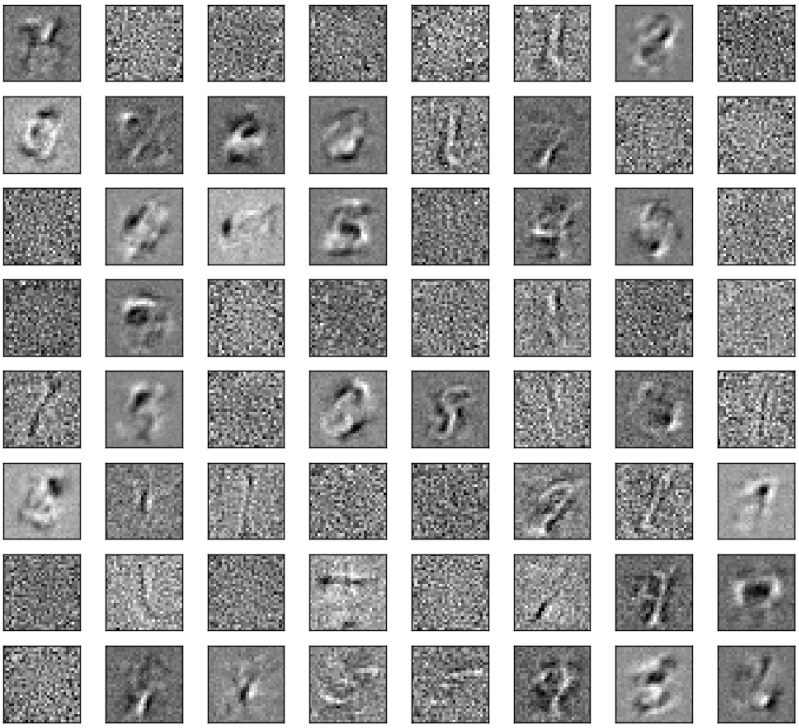


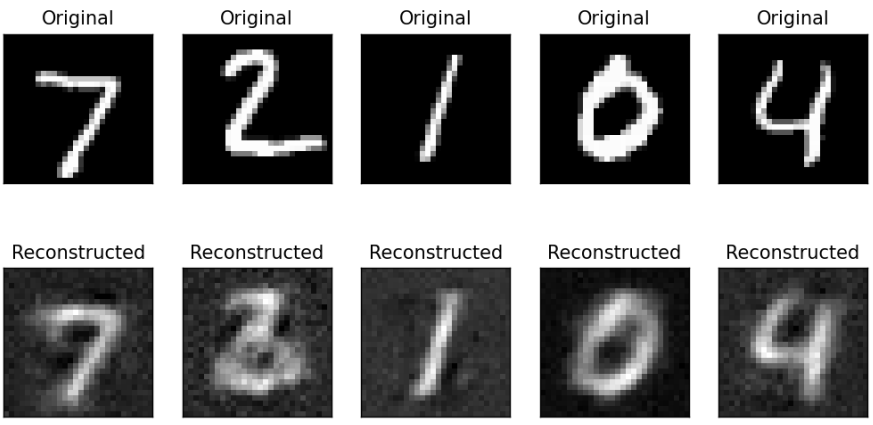


（1）使用随机 1000 个样本，步长为 0.000005，迭代次数为 10000

结果如下：

```
迭代 8500: 损失 = 0.10413547313784452
迭代 8600: 损失 = 0.1039788218275538
迭代 8700: 损失 = 0.10382543707703736
迭代 8800: 损失 = 0.10367503009908306
迭代 8900: 损失 = 0.10352777833921979
迭代 9000: 损失 = 0.10338366637607868
迭代 9100: 损失 = 0.10324235695969386
迭代 9200: 损失 = 0.10310389575811323
迭代 9300: 损失 = 0.10296815706957371
迭代 9400: 损失 = 0.10283517822769865
迭代 9500: 损失 = 0.10270466696756203
迭代 9600: 损失 = 0.10257664030534297
迭代 9700: 损失 = 0.1024510565960792
迭代 9800: 损失 = 0.10232809496806057
迭代 9900: 损失 = 0.10220777808214675
```





3. 分析和改进：

观察结果发现，随着迭代次数增加，可视化的特征矩阵更具有数字的“模样”。

改进：适当增加迭代次数，采取恰当的步长进行训练

五 实验小结：

TopoICA 模型是 ICA 模型的改进版，考虑了相临近成分之间的高阶依赖性，并通过一个拓扑顺序来定义成分之间的依赖性。在上述实验中比较成功的还原了输入数据，取得了较好的效果。